IBM hardware plan: predicate and QUBO QAOA for the rCRP

10 October 2026 · Radin's rCRP QAOA study · study folder QAOA_IBM_Hardware_Study_2026_10_10_v1 · no IBM job has been run · every number below is from simulation or circuit counting

> **Answer.** Yes, put **both** on IBM, but only where each one fits.
> - **Predicate QAOA:** 5 bays, a size ladder from 158 to 418 two-qubit gates.
> - **QUBO QAOA:** the 3 bays that fit under 400 two-qubit gates (the pilot, S3T4C6-24 and S3T4C7-32).
> - Same device, same day, same settings: 40 jobs, estimated 1.5–3.5 minutes of QPU time, hard cap 5 minutes.
>
> The real-chip run is optional for the paper, but it is the only way to test whether the noisy simulations are too optimistic.

1. Why both

- **Predicate is cheaper.** QUBO needs 1.5–3.6× more qubits and, after compiling for an IBM chip, more two-qubit gates on 15 of 15 bays.
- **Each method wins on a different measure.** Predicate gives more *legal* plans: in the perfect simulator on 4 of 5 bays, and with the IBM noise model clearly on 3 of 4 (equal on the 4th). QUBO gives more *optimal* plans: in the perfect simulator on 3 of 5 bays (equal on 2), and with the noise model on 4 of 4, though clearly only on the pilot; the other three gaps are within shot noise. Neither wins outright, so a same-bay hardware comparison directly tests your "qubits versus gates" question.
- **Only small bays fit.** Under 400 two-qubit gates there are only five small bays (6–7 containers) with predicate, and three with QUBO. Every 8–9-container bay is over the limit (418 to 3,252 gates), and the four "hard" bays need 1,086 to 3,252. That is not practical without error correction, and it is a limits result in its own right.

2. Experiment settings for each method

	Predicate QAOA	QUBO QAOA
Program file	`qaoa_implementations/qaoa_predicate.py`	`qaoa_implementations/qaoa_qubo.py`
Encoding	Galle CRP-I with presolve; exact rule indicators as Pauli-Z terms (rules ≤ 5 variables); no slack, no scratch	Galle CRP-I with presolve; squared penalties with binary slack bits
Bays (in run order)	S3T4C6-21 (pilot), S3T4C6-24, S3T4C7-32, S4T3C6-16, data3-3-8	S3T4C6-21 (pilot), S3T4C6-24, S3T4C7-32
Qubits / compiled two-qubit gates*	8–12 / 158–418	12–15 / 228–372
Angles (fixed, never retuned)	`loo:<bay>:p2`, leave-one-out over the 15 benchmark bays	`fresh:galle:p2`, chosen on 8 other QUBO bays; none is a ladder bay
Depth and circuit	p = 2. H on all qubits, then per layer the cost phase exp(–iγE/scale) and the RX(2β) mixer, then measure every qubit into register `decision` (bit i = variable i)	
Compiling	Recompile on the day for the live device: `generate_preset_pass_manager(optimization_level=3, seed_transpiler=11)`. Seeds 12 and 13 are recorded but not run. Submit only if check C (compiled = logical ideal output, ≤ 1e-9) passes on the live target.	
Shots	5 jobs × 2,000 shots = 10,000 per bay per method. Target: 95% interval for the legal-plan rate within ±0.01. Each job counts as one repetition.	
Primitive and options	Qiskit Runtime `SamplerV2` on a real device (no simulator). Dynamical decoupling OFF, gate and measurement twirling OFF, no readout correction. `max_execution_time` = 30 s per job.	
Controls (no QPU)	Same bay, same angles: exact ideal, compiled-ideal, FakeFez noise model, and matched 10,000-shot uniform random guessing, all through the same decoder and replay check.	

*FakeFez (156-qubit Heron r2 snapshot), seed 11. Counts on the live device will differ a little and are recorded on the day.

3. Conditions (both methods)

- **Device:** one Heron-class IBM device your account can use, picked before any result is seen. Both methods run on it the same day.
- **Order:** the pilot first, with predicate then QUBO, 10 jobs. Check only that the pipeline works (jobs finished, bit width, parsing, replay), not whether the results look good. Then the other bays, smallest first, alternating which method goes first (QUBO→predicate, then predicate→QUBO) so that slow device drift does not favour one method. The predicate-only bays come last.
- **Before you approve:** save your API key on the Mac only, then copy `config/approved_hardware_run.TEMPLATE.json` to `approved_hardware_run.json` with the device, the methods and a 300 s total cap. Without that file, hardware mode refuses to run.
- **Honesty rules:** no automatic resubmission. A failed job is recorded with its cost and replaced once, labelled. The known optimum is used only for scoring after the run. All jobs are reported, including bad ones. The paper calls it "hardware execution with simulation-trained angles", not hardware-trained QAOA.

4. File format

Stage	Files
Input per bay and method	Live-compiled circuit `prep/circuits[_qubo]/<bay>/isa_LIVE_<device>_seed11.qpy` with its check record `hardware/live_compile_<device>_<method>.json`; the approval JSON
Output per run	Folder `hardware/runs/<bay>__hardware` (predicate) or `<bay>__qubo__hardware`: `plan.json`, `backend_snapshot.json` (calibration), and per job `batch_k_receipt.json` (job id, saved before waiting), `batch_k_states.npy` (every shot in the order returned), `batch_k.json` (scores plus IBM usage), then `summary.json`
Tables	`results_predicate.csv` and `results_qubo.csv`: one row per bay × arm, with columns for legal and optimal rates, 95% intervals, spread between jobs, best and median relocations, exact ideal and uniform values, and `data_kind` = HARDWARE or SIMULATION
Usage log	`ledger/qpu_usage.jsonl`: job id and IBM-reported quantum seconds; a run stops if the next job could pass the cap

5. Results so far (simulation only), predicate / QUBO

Bay	Qubits	Compiled 2-qubit gates	Ideal legal	Ideal optimal	Noise model legal	Noise model optimal	Random legal
S3T4C6-21	8 / 12	158 / 228	0.249 / 0.246	0.087 / 0.127	0.192 / 0.170	0.067 / 0.084	0.047
S3T4C6-24	8 / 12	188 / 235	0.103 / 0.088	0.035 / 0.035	0.076 / 0.064	0.024 / 0.026	0.023
S3T4C7-32	9 / 15	237 / 372	0.115 / 0.110	0.038 / 0.048	0.084 / 0.071	0.026 / 0.030	0.023
S4T3C6-16	8 / 17	276 / 466	0.104 / 0.157	0.092 / 0.149	0.100 / 0.103	0.089 / 0.095	0.039
data3-3-8	12 / 22	418 / 632	0.042 / 0.037	0.013 / 0.012	0.028 / not run (22 qubits, over laptop limit)	0.005 / not run (22 qubits, over laptop limit)	0.006

Ideal values are exact probabilities. Noise-model values are FakeFez, 10,000 shots in 5 jobs. "Legal" = a decoded plan that passes the move-by-move replay; "optimal" = a legal plan with the fewest relocations. Both methods' circuits passed every validity check (they match the baseline maths to about 1e-16 and are unchanged by compiling). All 10 check files (5 bays × 2 methods) report passed.

6. Not yet known

- How a real chip does. IBM's noise model leaves out some real errors, so expect hardware to be worse.
- Exact QPU time. The 1.5–3.5 minute estimate is replaced by IBM's reported usage after the pilot.
- Why QUBO sometimes gives more optimal plans. These data do not show the cause.